

Laxu

Make your position capital efficient.

Version 0.1 · October 2026 · Robinhood Chain testnet (chain ID 46630)

Testnet only, no real funds. Laxu is an independent project built on Arcus, not affiliated with or endorsed by Arcus.

1. Abstract

When you open a leveraged trade, the money is locked until you close it.

Laxu turns each open perp trade on Arcus into its own token on Robinhood Chain, and gives every token an isolated lending pool. Holders borrow USDG against the live value of their trade while it keeps running. Once the creator lists a trade, others can buy into it, and the creator earns 2% on each buy-in.

The token prices itself on-chain from two reported inputs: the mark price and cumulative funding. The lending pool reads that live value, so a winning trade raises its own borrowing limit without anyone sending a transaction.

Status: the contracts are deployed on Robinhood Chain testnet. The core flow (open a real Arcus trade, receive the token, borrow against it, repay) runs end to end.

2. The problem

Who has it. Perp traders on Arcus, Robinhood Chain's perp venue. That includes its stock perps (TSLA, NVDA, SPY...).

What they do today. A trader with an open, profitable position who needs cash has two options:

- **close the trade**, which gives up the exposure they wanted; or
- **withdraw margin**, which shrinks the cushion and raises the effective leverage.

Either way, the value of the trade can't be used while the trade stays as it is.

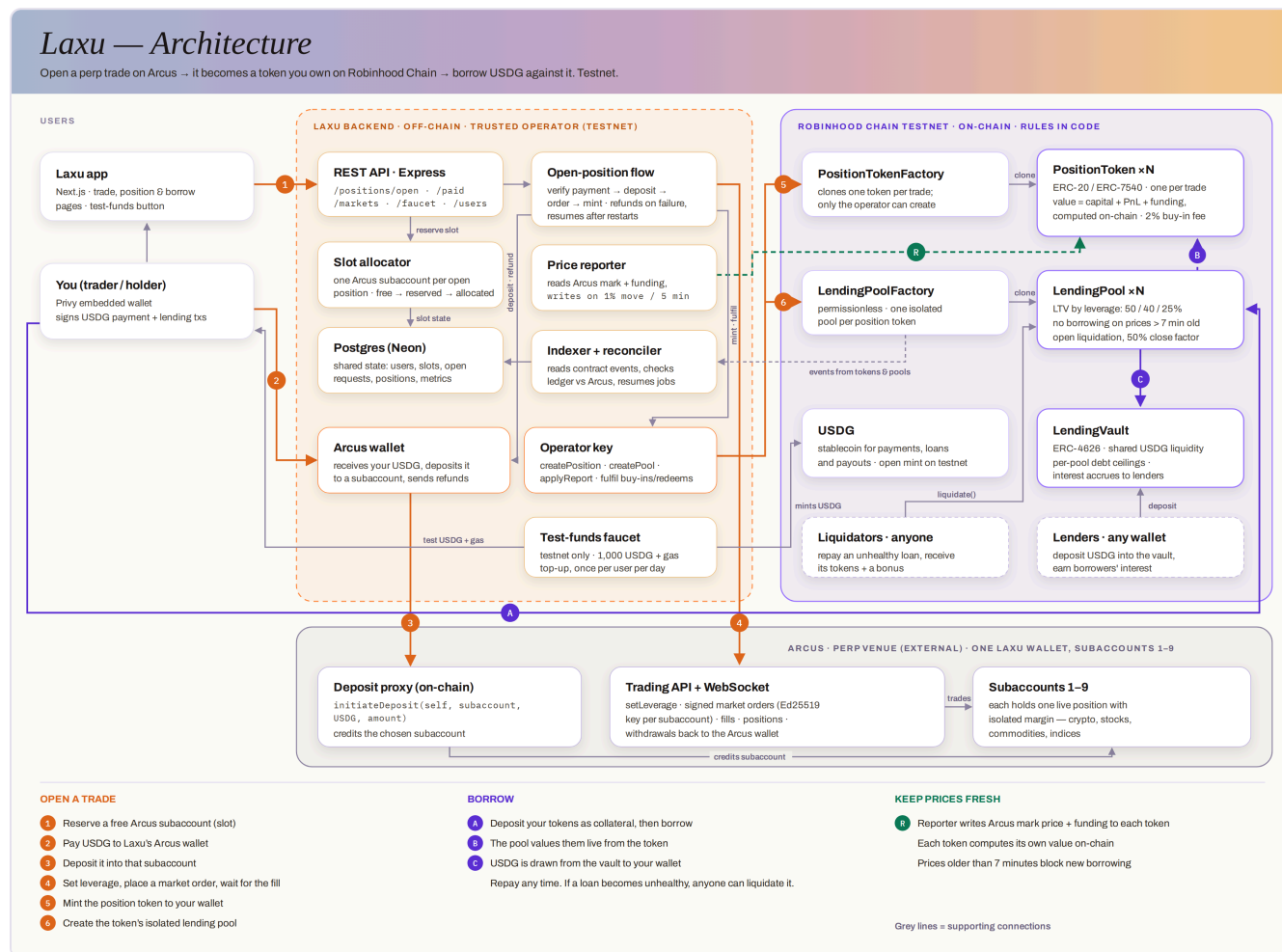
Why existing options don't cover it:

- **Venues** don't lend against open positions.
- **Lending markets** accept deposits and LP shares (for example, GMX GM tokens on Dolomite), not a trader's own position.
- **Leveraged tokens** (for example, Arcus pTokens) are products the platform defines, and there is no lending market for them yet.

Why now. Tokenized equities and equity perps are arriving on-chain (Robinhood Chain, Arcus), so a leveraged stock trade can now be collateral in an on-chain lending market.

3. How Laxu works

3.1 Architecture



The system has three zones.

- **The Laxu backend** (off-chain, a trusted operator) holds the Arcus wallet and the operator key. It runs the open-position flow, the slot allocator, the price reporter and an indexer/reconciler.
- **Robinhood Chain** (on-chain) holds the rules: `PositionTokenFactory` and one `PositionToken` per trade, `LendingPoolFactory` and one `LendingPool` per token, and a shared `LendingVault`.
- **Arcus** (an external venue) holds the actual perp position in a subaccount.

The backend connects the two systems. The contracts limit what it can do (§4.3).

3.2 Opening a trade

1. **Reserve a slot.** The backend reserves one of its Arcus subaccounts for the user, before any money moves.
2. **Pay.** The user sends USDG from their Privy wallet to Laxu's Arcus wallet.
3. **Deposit.** The Arcus wallet calls `initiateDeposit` on Arcus's deposit proxy, crediting the reserved subaccount.
4. **Trade.** The backend sets leverage, places a market order and waits for the fill.
5. **Mint.** The operator calls `PositionTokenFactory.createPosition` with the fill details. The token is minted to the user.

6. **Pool.** The operator calls `LendingPoolFactory.createPool`, which creates the token's isolated lending pool.

On testnet this takes about 1–2 minutes, most of it waiting for the deposit credit.

3.3 Borrowing

- **A.** The user deposits position tokens into the token's `LendingPool` as collateral.
- **B.** The pool values them live from the token (`totalAssets` / NAV per token), which moves with each price report.
- **C.** USDG is drawn from the shared `LendingVault` and sent to the user. They can repay any time.

Worked example (a 1–5× position: 50% max LTV, 60% liquidation threshold):

- Open **200 USDG at 5×** (1,000 USDG notional). The user can borrow **100 USDG** straight away.
- The position gains **+50 USDG**. It is now worth 250, and the borrowing limit rises to **125 USDG** with no transaction from anyone.
- Suppose instead they borrowed 100 and the position loses value. Once debt exceeds **60%** of the collateral value (a collateral value below about 166.7 USDG, a 3.3% adverse price move at 5×), anyone can **partially liquidate** the loan.

3.4 What the open flow has to get right

Two systems, one action. Payment and minting happen on-chain. The deposit, leverage setting and order happen on an off-chain venue. A crash between any two steps must not lose or double-spend money.

The flow is a resumable state machine:

```
awaiting_payment → payment_received → deposited → order_filled → minted
                  ↘ refunding → refunded
```

Each step saves what it learned (transaction hashes, credited amounts, fill details) before moving on, and is safe to retry. After a restart, the backend resumes every unfinished request from its saved state. Any failure before the fill refunds the user. If the funds were already deposited, the refund goes through an Arcus withdrawal back to the Arcus wallet and then to the user. Payments from the wrong sender or to the wrong wallet aren't accepted. Payments of the wrong amount, and payments that arrive after the reservation expired (15 minutes), are refunded.

Venue constraints that shaped the design:

- **Arcus deposits require** `owner == caller`. A user can't deposit into an account Laxu controls, so users pay Laxu's Arcus wallet, which deposits for them.
- **An Arcus API key binds to exactly one wallet plus one subaccount.** Each open position therefore gets its own subaccount (a "slot") and its own Ed25519 signing key. Slots move through `free → reserved → allocated`, and an expiry sweep frees abandoned reservations.
- **10 subaccounts per wallet** (indexes 0–9). Index 0 is where leftover balances are swept when a slot is freed, so 1–9 are trading slots: one wallet carries up to 9 concurrent positions, one per subaccount whose API key has been provisioned. Capacity grows by provisioning more keys or adding wallets; nothing else changes. When every slot is busy, a new open is refused before the user pays.
- **Market orders need a protective price within 10% of mark**, and `goodTilTime` is mandatory, even on IOC orders. Laxu uses a 9% bound (`ARCUS_SLIPPAGE_BPS = 900`), and config validation rejects anything above 10%.
- **Leverage must be set before the first order** on a subaccount.

Units across three systems:

Quantity	Unit
Prices (entry, mark, fill) on-chain	$\times 1e18$
Position size	base quantity $\times 1e6$, always positive (direction carries the sign)
USDG amounts and position-token shares	6 decimals
Arcus withdrawal amounts	quote quantums, $1e9 = \$1$
Funding	signed USDG, 6 decimals, positive = received

Size at $1e6$ is what makes $\text{size} \times (\text{mark} - \text{entry}) / 1e18$ land in USDG's 6 decimals. For example, 0.25 ETH on a \$200 move gives $250,000 \times 200e18 / 1e18 = 50,000,000$, which is \$50. A unit error here doesn't throw; the numbers just come out wrong. So every conversion lives in one module, `Backend/src/lib/units.ts`. It never uses JavaScript floats for money and has its own tests.

4. Mechanism design

4.1 PositionToken: one contract per trade

Each trade is an **EIP-1167 minimal-proxy clone** of one `PositionToken` implementation. It has its own supply, balances and parameters: market, direction, leverage, entry price, size and capital.

Valuation, fully on-chain:

```
value = capital + size × (mark - entry) / 1e18    (PnL negated for shorts)
      + (funding - fundingSettled)
NAV per token = value / totalSupply
```

The backend reports only two numbers per update: the mark price and cumulative funding, through `applyReport`. Report timestamps must strictly increase. The contract derives everything else, so the operator has no function that sets a share price directly.

Asynchronous requests (ERC-7540). A buy-in or redeem requires changing the Arcus position, which happens off-chain and takes time. So buy-ins (`requestDeposit`) and redeems (`requestRedeem`) are ERC-7540 asynchronous requests, which the operator fulfils after the Arcus side has moved.

- **Auto-settle on fulfil.** The buyer receives tokens, or the redeemer receives USDG, in the fulfil transaction itself. There is no separate claim step for the user.
- **A 20-minute cancel window.** If a request isn't fulfilled within 20 minutes, the requester can cancel it: a buy-in returns its pending USDG (the 2% fee was already paid to the creator), and a redeem re-mints its shares. Once a position closes, a pending buy-in can be cancelled immediately.

Proportional accounting. With current value V and supply S :

- a buy-in of a USDG mints $\Delta S = a \times S / V$ shares, priced at the NAV of the latest report. Size grows by the size actually added on Arcus (the buyer's proportional share), entry moves to the size-weighted average of old entry and fill price, and capital grows by a .
- a redeem of fraction $f = \text{shares} / S$ pays $f \times V$. Size shrinks by the size actually closed on Arcus, entry is unchanged, and capital and settled funding scale by the same f .

Both operations keep **NAV per token the same for every holder**, and keep leverage the same as long as Arcus can fill the proportional change. If that change is below the market's minimum order size, the operator passes a size of 0: a buy-in then backs the position as extra margin only, and a redeem is paid out of margin, so leverage drifts slightly. This is also why leverage can't change after a trade is tokenized: a token's leverage is a property shared by every holder, not a per-holder setting. When the last share leaves, by redeem or trigger, the position closes and settles at zero by itself.

Lifecycle rules:

- `list(nickname)` is creator-only and one-way. It opens the position to outside buy-ins. Nicknames are capped at 32 bytes.
- **Buy-in fee:** 2% (`BUY_IN_FEE_BPS = 200`) of each non-creator buy-in goes to the creator. It is charged at request time and isn't refunded if the request is cancelled. The creator's own top-ups are fee-free.
- `requestClose` only works when the creator holds 100% of the supply and no request is pending.
- **Close → settle → claim.** The operator closes the Arcus position and records the final mark and funding with `close()`, which fixes an estimated final value. It then withdraws what Arcus returns and calls `settle(assets)` with the amount actually recovered, which can be below the estimate after a liquidation penalty. The contract checks that the token holds that amount (plus any pending buy-ins, which stay refundable). Each holder then claims their pro-rata share, or anyone can call `claimFor` for an EOA holder, which always pays the holder, never the caller. The backend pushes claims to every EOA holder.
- **Liquidation on Arcus** is recorded as `close(..., true)`, and the position settles at whatever value remains, including zero.

Per-holder stop-loss / take-profit. The creator's levels are the default for every holder, and each holder can override or clear them. When a holder's level is hit, the operator exits only that holder's wallet balance at NAV, leaving the other holders' NAV and leverage unchanged. A trigger never touches tokens posted as lending collateral. The contracts are done and tested. The backend watcher and the position-page panel are built but **not yet verified end to end on testnet**.

4.2 Lending: a shared vault with isolated pools

LendingVault (ERC-4626). Lenders deposit USDG in one place. Lent-out USDG still counts in `totalAssets`, so a borrow doesn't move the share price, and repaid interest raises it. Each pool has its own debt ceiling at the vault, and only registered pools can draw from it.

LendingPool, one per position token. Pools are created by a permissionless factory. Risk parameters are fixed at creation from the token's own leverage, and **the caller sets none of them**. Two pools for the same token get identical parameters.

Leverage	Max LTV	Liquidation threshold	Liquidator bonus
1–5×	50%	60%	8%
6–10×	40%	50%	10%
11–20×	25%	35%	12%

Higher leverage moves value faster, so it borrows less and pays liquidators more to clear it quickly. These numbers are a starting point and haven't been back-tested.

Health factor = collateral value × liquidation threshold / debt. Collateral value is read **live** from the token, so profit raises the limit automatically and losses lower it. At the LTV cap, the health factor is threshold / LTV (1.2 for the 1–5× tier), which is the cushion before liquidation.

Liquidation:

- It is permissionless: any address can call `liquidate()`.
- The close factor is 50% of the debt per call. It rises to 100% when the health factor is below 0.95, or when the remaining collateral is dust (under \$50).
- The bonus is paid in position tokens, transferred directly to the liquidator. It doesn't go through the asynchronous redeem path.
- When a position is deeply underwater, the seizure is capped at the collateral actually held, so the loan stays liquidatable instead of reverting.
- Example (1–5× tier): with 100 USDG of debt, a liquidator repays 50 and receives 54 USDG worth of position tokens at NAV.

Interest is a flat 10% APR, simple interest (`BORROW_APR_BPS = 1_000`), on testnet. A utilization curve prices the scarcity of liquidity within one market, but here liquidity is shared across every pool, so a single pool's utilization isn't a meaningful input. **(Planned)** a rate based on the vault's utilization, with a protocol reserve share.

Limits: bad debt isn't absorbed. If seized collateral can't cover a loan, the vault's `totalAssets` stays overstated by the shortfall, and there's no insurance fund yet. The tier table also has no upper bound: leverage above 20× falls into the 11–20× tier. Laxu's backend caps leverage at 20×, but `LendingPool` doesn't enforce it.

4.3 Price freshness: an asymmetric rule

The reporter checks every open position's Arcus mark and funding once a minute. It writes an on-chain report when the mark has moved **1%** since the last report, or when **5 minutes** have passed (the heartbeat).

`MAX_REPORT_AGE = 7 minutes`: the 5-minute heartbeat plus a 2-minute buffer for execution and confirmation.

The rule is asymmetric:

- **Stale prices block actions that open new risk:** `borrow()` and `withdrawCollateral()`.
- **Stale prices never block** `liquidate()` or the `healthFactor()` view. Liquidating on last-known data is better than leaving bad debt open while nobody can act.
- **Closed positions are exempt.** Reports stop at close, so without the exemption a borrower could never withdraw collateral to claim. (`repay()` is never gated.) Borrowing against a closed position is refused separately.

What the operator key can and can't do. Every number the operator gives the contracts comes from Arcus:

- **It supplies:** mark and funding reports (with strictly increasing timestamps); the fill size and fill price when it fulfils a buy-in, redeem or trigger; the final mark and funding at `close()`; and the recovered amount at `settle()`.
- **It cannot:**
 - set a share price directly (NAV is always derived by the formula above);
 - mint shares to itself or change anyone's balance outside a fulfilled request;
 - exit a holder whose own trigger level isn't hit at the stored mark;
 - pay a claim to anyone but the holder;
 - touch pending buy-in USDG or unclaimed settlement funds (`recoverExcess` only returns the balance above both, which is the float the backend fronted);
 - block `repay()`, `liquidate()` or `claim()`.

Limit: the freshness rule guards against *old* data, not *wrong* data. A dishonest or faulty operator could misreport a price, a fill or the settlement amount, and the contract would only check that the payout is funded (\$6).

5. The invariants we hold

Each row is a property the system relies on, the mechanism that enforces it, and the test that checks it. Test names are quoted exactly from `Contracts/test/`.

Invariant	Enforced by	Test
A buy-in never changes NAV per token	Proportional mint, size-weighted entry	"a buy-in at NAV 1.48 leaves NAV at 1.48 and grows size, entry and capital"
A redeem pays exactly its share and keeps NAV	Fraction-scaled capital and settled funding	"redeeming 50% pays 50% of totalAssets, halves capital and effective funding, and keeps NAV"
Profit raises the borrowing limit with no transaction	Live valuation from the token	"hands the borrower more headroom automatically when the position gains value"
Stale prices can't create new debt	<code>freshOracle</code>	"blocks borrow() once the collateral's last report is older than MAX_REPORT_AGE"
Stale prices never block liquidation	Freshness asymmetry	"does NOT block liquidate() on stale data -- liquidating on last-known data beats not liquidating at all"
Liquidation is bounded and permissionless	Close factor plus a fixed bonus, no role check	"liquidates at 50% close factor and pays the 8% bonus in shares"; "is permissionless -- an address holding no protocol role can liquidate"
Payouts stay fair until the last claim	<code>totalAssets</code> / <code>totalSupply</code> held constant during claims	"pays 1,000 USDG out 500/300/200 and keeps totalAssets/totalSupply constant between claims"
Collateral can't be pulled by a trigger	Triggers exit wallet balances only	"only exits the wallet balance, not tokens posted as LendingPool collateral"
The creator can't close a position backing a loan	Close requires holding 100% of the supply	"blocks the creator's requestClose while their shares are posted as collateral"

Totals: 114 contract tests (76 for `PositionToken`, 38 for lending) and **90 backend unit tests** covering units, sizing, signing, settlement, trigger math, faucet rules, the open-position flow and market data.

```
cd Contracts && npx hardhat test
cd Backend && npm test
```

6. Trust model and tradeoffs

The on-chain parts are trustless, and the parts that touch Arcus aren't yet. Here is where trust sits today, and how it can be removed.

Component	Today (testnet)	Why	Path to remove trust
Arcus positions	Custodial: Laxu's wallet holds the subaccounts	Arcus requires the depositor to own the account	Per-user or contract-owned Arcus accounts (planned)
Price reports	Laxu backend (a single reporter)	Chainlink CRE costs about \$600 and access is gated	CRE fetching Arcus's mark directly, plus a Chainlink price-feed sanity bound (planned)

Component	Today (testnet)	Why	Path to remove trust
Fills and settlement amounts	Reported by the Laxu backend; the contract checks only that payouts are funded	Arcus fills and withdrawals happen off-chain	Verifiable venue data (e.g. CRE reading fills and balances) (planned)
Liveness	A single backend	Speed of shipping	Multiple reporters for uptime; keeper incentives (planned)
Lending	On-chain , permissionless liquidation	Lenders' money sits here, so we removed trust from this component before any other	—
Capacity	Up to 9 concurrent positions per Arcus wallet, one per provisioned subaccount key	Venue limit (10 subaccounts per wallet, index 0 kept for sweeps)	Provision more keys; add wallets

What happens when the backend fails? Lending keeps working. Users can repay at any time, liquidation stays open to anyone, and a closed position's borrowers can always withdraw their collateral. After 7 minutes without reports, new borrowing and collateral withdrawals on open positions pause until reports resume. An unfulfilled buy-in can be cancelled after 20 minutes for its pending USDG (the 2% creator fee isn't returned), and an unfulfilled redeem for its shares. An open that crashed mid-flow resumes on restart, or refunds if it failed before the fill. Settlement and claims after a close need the backend to bring funds back from Arcus. Not handled yet: an Arcus outage that outlasts the timeouts during an open needs manual resolution.

The contracts limit the operator to supplying Arcus data (§4.3). A dishonest operator could misreport a price, a fill or a settlement amount, but it can't mint itself shares, take pending buy-ins or unclaimed settlement funds, or send a payout to anyone but the holder.

7. Decisions and what we cut

Borrowing, not copy trading. Copy trading is a crowded idea, and lending against an open trade is the need no existing product meets. Buy-ins remain, because they support borrowing: they give a position token buyers, and backers can borrow against their own slice.

A token, not a platform loan. A loan from Laxu's own balance sheet would make Laxu a bank: our money, our database, trust us. A token puts the collateral on-chain, so anyone can lend through the vault, anyone can liquidate, the rules are code, and buy-in holders can borrow too. *Cost:* every open now has an off-chain leg and an on-chain leg that must agree, which is why §3.4 exists.

Isolated pools over one shared risk pool. One bad position can't create losses in another pool. Lenders still get a single vault, and each pool has its own debt ceiling. *Cost:* no cross-collateral borrowing.

A backend reporter instead of Chainlink CRE (for testnet). This avoided the cost and access gate, and the contract rules bound what the reporter can do. *Cost:* a trusted price source until mainnet (§6).

What we cut, and why:

- **The liquidation bot** is built but off by default. `liquidate()` is permissionless on-chain, so the bot is a convenience, and leaving it off saves operator gas on testnet.
- **A vault-lending screen.** Lenders can use the standard ERC-4626 vault directly. On testnet the deployer seeds it.
- **A secondary market.** Tokens are standard ERC-20s, so it can come later.
- **A community feed.** Not built on real data.

All of these were cut to ship one flow that works end to end: **open** → **borrow**.

8. Market and positioning

Three adjacent categories:

- **Perp venues** (GMX, Hyperliquid, Arcus): a trade can't be used as collateral until it closes.
- **Pooled liquidity tokens** (GMX GM): they tokenize the house's side of the trade, not the trader's.
- **Leveraged tokens** (Arcus pTokens, Toros, Index Coop): products the platform defines, with no lending against them yet.

The 2×2:

	Can't borrow against it	Can borrow against it
Platform-made product	Leveraged tokens (pTokens, Toros, Index Coop)	Pooled LP tokens (GMX GM on Dolomite)
Your own trade	Perp venues (GMX, Hyperliquid, Arcus)	Laxu

Among the products we reviewed, Laxu is the only entry in "your own trade × can borrow against it".

Evidence the pattern works. GMX GM tokens are already used as collateral on Dolomite. They are minted through the same request-then-execute flow and get the same isolated-collateral treatment. Laxu applies that pattern to the trader's side of the trade.

Expansion: lending markets for Arcus pTokens (**planned**), then other venues. The token and pool contracts don't depend on Arcus; only the backend adapter does.

9. Go-to-market

- **Start with** Arcus traders on Robinhood Chain, particularly stock-perp traders, for whom "borrow against my TSLA trade" is a new option.
- **Grow through** shareable trade pages. Creators earn 2% on each buy-in, which gives them a reason to share their positions.
- **Cold start:** Laxu seeds the vault so borrowers have liquidity from day one. Lenders earn the borrowers' interest.
- **Launch safely:** mainnet with low per-pool debt ceilings and caps, raised as the system proves itself.
- **Channels:** Robinhood Chain and Arbitrum builder communities, X Spaces, events.
- **Arcus partnership:** a goal, not a current relationship.

10. Business model

Today: the 2% buy-in fee goes to creators, and interest goes to lenders. **The protocol doesn't earn anything yet.**

(Planned):

- A **protocol share of interest** (a reserve factor, as Aave uses, typically 10–15%), which would also fund a bad-debt reserve (§4.2).

- A **split buy-in fee or a profit share**. For reference, copy-trading leaders typically take 10–20% of profits, and Hyperliquid vault leaders take 10%.

11. Roadmap

1. **Mainnet price safety**: Chainlink CRE fetching Arcus's mark, a price-feed sanity bound, multiple reporters.
2. **Less custody**: per-user or contract-owned Arcus accounts.
3. **pTokens lending**: isolated pools for Arcus pTokens.
4. **Secondary market and social layer**: trading position tokens, plus a real community feed.
5. **More venues**: Hyperliquid, Monad.

12. Risks and disclosures

- **Leverage risk**. Leveraged positions can lose all their capital quickly.
- **Liquidation risk**. A loan whose health factor falls below 1 can be liquidated by anyone, at a bonus of 8–12% paid from the borrower's collateral.
- **Venue risk**. Arcus outages, resets or rule changes affect every position. Testnet venues can reset.
- **Oracle risk**. Prices come from a single trusted reporter today (§6).
- **Smart contract risk**. The contracts are **unaudited**.
- **Regulatory risk**. Tokenized leveraged positions may count as derivatives in some jurisdictions. Restricted regions will follow on-chain perp norms, and we'll get a legal review before mainnet.

Testnet only, no real funds. Independent project built on Arcus, not affiliated with or endorsed by Arcus.

13. Credits and how it was built

What we started from:

- **OpenZeppelin Contracts**: ERC-20, ERC-4626, Clones, SafeERC20, access control.
- **OpenZeppelin community contracts**: `ERC7540`, `ERC7540AdminDeposit` and `ERC7540AdminRedeem`, **vendored and modified** in `Contracts/contracts/vendor/`. Four storage fields (`_deposits`, `_redeems`, `_totalPendingDepositAssets`, `_totalPendingRedeemShares`) were changed from `private` to `internal`, so `PositionToken` can auto-settle on fulfil. Each file's header records the change and the hash of the unmodified upstream file.
- **Also**: the Arcus testnet API, WebSocket and deposit proxy; Privy (auth, embedded wallets); viem; Prisma with Neon Postgres; Next.js; TradingView Lightweight Charts (Apache 2.0).

Everything else (the position token, the lending contracts, the backend and the frontend) was written for Laxu.

Where the key decisions live:

File	Decision
<code>Contracts/contracts/PositionToken.sol</code>	On-chain valuation, proportional buy-in/redeem, close/settle/claim, per-holder triggers

File	Decision
<code>Contracts/contracts/LendingPool.sol</code>	Risk tiers, <code>freshOracle</code> asymmetry, close factors, interest
<code>Contracts/contracts/LendingVault.sol</code>	Shared liquidity, per-pool debt ceilings
<code>Backend/src/services/openPosition.ts</code>	The resumable open flow and refunds
<code>Backend/src/services/allocator.ts</code>	Subaccount slots
<code>Backend/src/services/reporter.ts</code>	The 1% / 5-minute reporting policy
<code>Backend/src/lib/units.ts</code>	Every unit conversion

How it was built. Laxu was built by one developer with an AI coding assistant (Claude Code). The product decisions, architecture, risk parameters and tradeoffs in this paper are ours. All generated code was reviewed, tested and corrected where it was wrong for our case. One example: the stale-price guard was originally applied to closed positions too. Since reports stop at close, borrowers on a closed position could never have withdrawn their collateral to claim their payout. We added the closed-position exemption and a test for it (*"exempts a closed position: repay + withdrawCollateral work long after close, borrow does not"*).

Appendix

A. Parameters

Parameter	Value	Where
Max LTV / liquidation threshold / bonus, 1–5×	50% / 60% / 8%	<code>LendingPool._riskTierFor</code>
Max LTV / liquidation threshold / bonus, 6–10×	40% / 50% / 10%	<code>LendingPool._riskTierFor</code>
Max LTV / liquidation threshold / bonus, 11–20×	25% / 35% / 12%	<code>LendingPool._riskTierFor</code>
Close factor	50% of debt per call	<code>CLOSE_FACTOR_BPS = 5_000</code>
Full-liquidation health factor	below 0.95 → 100% close factor	<code>CLOSE_FACTOR_HF_THRESHOLD = 0.95e18</code>
Dust threshold	under \$50 of collateral → 100% close factor	<code>DUST_THRESHOLD_USD = 50e6</code>
Borrow rate	10% APR, simple interest	<code>BORROW_APR_BPS = 1_000</code>
Max report age	7 minutes	<code>MAX_REPORT_AGE</code>
Reporter check interval	60 seconds	<code>REPORTER_INTERVAL_MS</code>
Reporter deviation trigger	1%	<code>DEVIATION_THRESHOLD_BPS = 100</code>
Reporter heartbeat	5 minutes	<code>HEARTBEAT_SECONDS</code>
Buy-in fee	2% of non-creator buy-ins, to the creator	<code>BUY_IN_FEE_BPS = 200</code>
Request cancel window	20 minutes	<code>REQUEST_CANCEL_TIMEOUT</code>
Nickname length	32 bytes max	<code>MAX_NICKNAME_LENGTH</code>

Parameter	Value	Where
Market-order protective bound	9% from mark (Arcus max is 10%)	<code>ARCUS_SLIPPAGE_BPS = 900</code>
Slot reservation timeout	15 minutes	<code>RESERVATION_TIMEOUT_MS</code>
Leverage cap	20× (backend only; the contracts don't enforce it)	<code>LAXU_MAX_LEVERAGE</code>
Slots per Arcus wallet	up to 9 (subaccounts 1–9; index 0 receives sweeps)	<code>provisionSlots.ts</code> , <code>sweep.ts</code>

B. Deployed addresses

Robinhood Chain testnet, chain ID 46630. Explorer: <https://explorer.testnet.chain.robinhood.com>

Contract	Address
PositionToken (implementation)	<code>0x60101F14631bAAff60f09D5BA0aDF3F940d15e2a</code>
PositionTokenFactory	<code>0x15F8DFF61656e17a5C5AE7e03571f9833bBEb84c</code>
LendingVault	<code>0x6Defff9515D183AB7bBd3BBcE822C54D852D4bEf</code>
LendingPool (implementation)	<code>0x8b8d3EeE0BF4f42417C2b4d14491D343F162Bd1C</code>
LendingPoolFactory	<code>0x6DDb43385fbB07a5c9ee2b84343940eE84f138ab</code>
USDG	<code>0x293b337712d4312776a3a2d292f44410e7873bad</code>

C. Glossary

- **Perp (perpetual future):** a leveraged contract that tracks an asset's price and has no expiry.
- **Mark price:** the venue's reference price for valuing positions and triggering liquidations.
- **Funding:** periodic payments between longs and shorts that keep the perp price close to the underlying asset's price.
- **Leverage:** position notional divided by capital. 5× means 200 USDG controls 1,000 USDG of exposure.
- **LTV (loan-to-value):** debt divided by collateral value. Max LTV is the most you can borrow.
- **Health factor:** collateral value × liquidation threshold / debt. Below 1, the loan can be liquidated.
- **Liquidation:** repaying part of an unhealthy loan in exchange for the borrower's collateral plus a bonus.
- **NAV (net asset value) per token:** the position's value divided by the token supply.
- **ERC-4626:** the standard interface for tokenized vaults, where deposits mint shares.
- **ERC-7540:** an extension of ERC-4626 for asynchronous deposits and redeems (request, then fulfil).
- **Subaccount:** an isolated trading account under one Arcus wallet, with up to 10 per wallet.